Homework 1: Backpropogation and Neural Networks

Deep Learning (84100343-0)
Autumn 2023
Tsinghua University

1 Part One: Backpropogation

1.1 Introduction

Squeeze-and-Excitation Networks (SENets) [2] won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2017 [9]. SENet fundamentally addresses a critical issue in deep learning: how to better allocate and emphasize important features while reducing the impact of less informative ones. This challenge arises because traditional CNNs often treat all channels or feature maps equally during convolutional operations, leading to suboptimal feature selection. SENet tackles this problem by introducing a novel architecture that explicitly models interdependencies between feature channels.

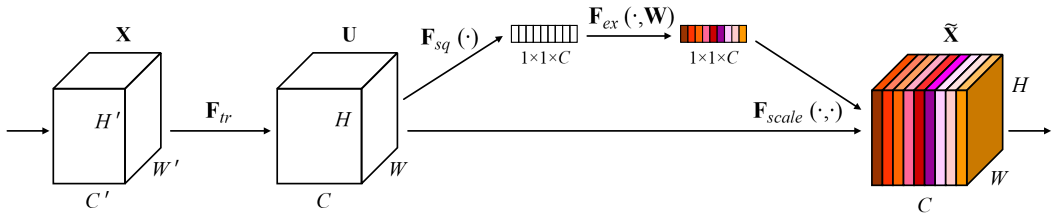

Figure 1: A Squeeze-and-Excitation block in SENets [2].

As shown in Figure 1, SENet employs a two-step mechanism: *squeezing* and *excitation*. During the squeezing step, it compresses the spatial information within each feature map into a single channel, effectively summarizing the information content. Then in the excitation step, it learns channel-wise weights that represent the importance of each feature map. These learned weights are then applied to the original feature maps, allowing SENet to adaptively recalibrate the features in a context-aware manner. This process significantly enhances the network's ability to focus on the most relevant features while suppressing less useful ones, ultimately improving its performance in various deep learning tasks. SENets formed the foundation of the winner of ILSVRC 2017, which reduced the top-5 error to 2.251%, surpassing the winning entry of 2016 by a relative improvement of $\sim$25%.

1.2 Network Details

Figure 2 presents a simplified squeeze-and-excitation network for this homework. For simplicity, we only use fully connected layers and leave out convolutional and pooling layers.

For a mini-batch of training samples $(\mathbf{x}, \mathbf{y})$ with a batch size of m:

1. For each input sample $\mathbf{x}^i \in \mathbb{R}^d$ in a mini-batch, $\mathbf{y}^i$ is a one-hot vector for classification task with K classes.
2. As illustrated by the boxes colored in red, FC_1, FC_2, FC_3 and FC_4 are fully-connected layers, in which FC_1, FC_2, FC_3 serve as hidden layers with h neurons, while FC_4 are output layers with K neurons.

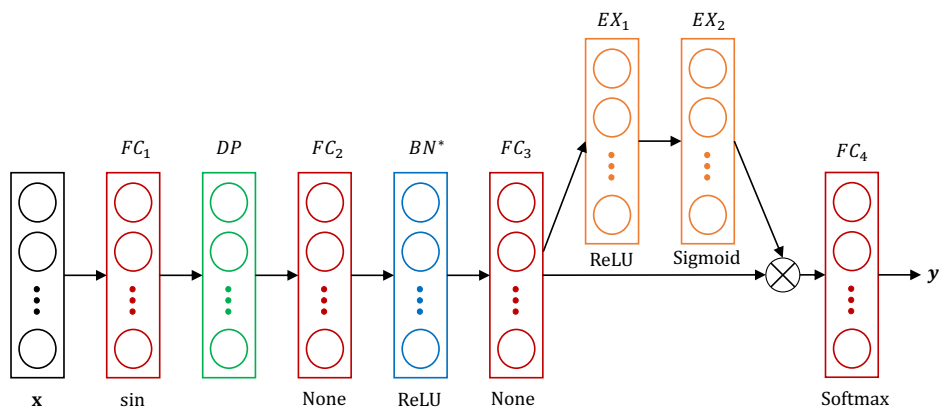

Figure 2: Schematic of the simplified squeeze-and-excitation network in this homework.

3. As illustrated by the orange boxes, EX_1 and EX_2 in the excitation mechanism are also fully-connected layers, which have s and h neurons, respectively. Outputs of EX_2 are fed into a Sigmoid activation function to ensure the feature selection weights range from 0 to 1.
4. Denote weight matrixs of all fully connected layers as Θ_1, Θ_2, Θ_3, Θ_4, Θ_{ex1} and Θ_{ex2}, whose bias vectors are $\mathbf{b}_1$, $\mathbf{b}_2$, $\mathbf{b}_3$, $\mathbf{b}_4$, $\mathbf{b}_{\mathrm{ex1}}$ and $\mathbf{b}_{\mathrm{ex2}}$ respectively. Note that, FC_2 is slightly different from the standard fully connected layer, detailed in 7. below.
5. As illustrated by the green box, DP is the dropout layer with a random mask vector $\mathbf{M}$. In the training stage, the probability that a neuron of FC_1 will be dropped is denoted as p. Formally,

$$\mathbf{M}_j = \begin{cases} 0, & r_j < p \\ 1/(1-p), & r_j \geq p \end{cases},$$

   where r_j is a random value between 0 and 1 for the jth neuron.
6. FC_1 uses the *sine* as a periodic activation function, proposed by a recent paper [12] published in NeurIPS 2020 as an oral presentation. Neural networks with this are demonstrated to fit complicated signals, such as natural images and 3D shapes, and their derivatives robustly.
7. As illustrated by the blue box, BN^* is a mean-only batch normalization layer [10] for FC_2. Different from the standard batch normalization [4], inputs are only subtracted by the mini-batch means, without being divided by the mini-batch standard deviations. Formally, let $\mathbf{x}_{\mathrm{DP}}$ denote the output of the dropout layer,

$$\begin{aligned} \mathbf{x}_2 &= \Theta_2 \mathbf{x}_{\mathrm{DP}} \\ \mu &= \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}_2^i \\ \mathbf{x}_{\mathrm{BN}} &= \mathbf{x}_2 - \mu + \mathbf{b}_2. \end{aligned} \tag{1}$$

   After that, $\mathbf{x}_{\mathrm{BN}}$ is fed into a ReLU activation function.
8. For each sample, the notation $\bigotimes$ means *element-wise multiplication* of two vectors.
9. For each input sample $\mathbf{x}^i$, the final prediction is $\widehat{\mathbf{y}}^i$. We use the **cross-entropy** loss:

$$\mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}\left[-\sum_{k=1}^{K} \mathbf{y}_k^i \log \widehat{\mathbf{y}}_k^i\right]. \tag{2}$$

1.3 Task Description

For this part, please answer the following questions and submit a PDF (handwriting, Word, or LaTeX) with detailed derivation and computation.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Figure 3: Batch Normalization Transform, applied to activation x over a mini-batch.

1. Given a standard BatchNorm layer, please calculate the gradients of the **output** $y_i = \mathbf{BN}_{\gamma,\beta}(x_i)$ with respect to the **parameters** of γ, β shown in Figure 3. (**5 points**)
2. Given a softmax function, please calculate the gradients of **the output of a softmax function** with respect to **its input**. (**5 points**)
3. Finish the detailed **feed-forward computations** of a batch of samples $(\mathbf{x}, \mathbf{y})$ during a training iteration, coming with final predictions $\widehat{\mathbf{y}}$. (**10 points**)
4. Use the backpropagation algorithm we have learned in class and give **the gradients of the overall loss in a mini-batch with respect to the parameters at each layer**. (**15 points**)

Note that:

1. Please show the necessary derivations of the gradients.
2. Please attach variable notations in the gradients expressions.
3. A vectorial form of gradient expressions is highly encouraged.
4. In the spirit of backpropagation, you can **use gradients of some parameters, in addition to inputs, outputs, and parameters, to represent gradients of other parameters**.

2 Part Two: Multilayer Perceptron (MLP)

Figure 4 presents a 3-layer MLP. In this part, you are required to implement and train this 3-layer neural network to forecast time series from the ETTh1 (Electricity Transformer Temperature) dataset [19], where each series consists of the target value "oil temperature" and 6 power load features.

2.1 Time Series Forecasting

We formulate the multivariate time series forecasting problem as follows. We denote historical observations of each data sample as $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_T\} \in \mathbb{R}^{T\times N}$ with T timesteps and N variates, and aim to predict the future S timesteps $\mathbf{Y} = \{\mathbf{y}_{T+1}, \ldots, \mathbf{y}_{T+S}\} \in \mathbb{R}^{S\times N}$. Recent literature [18] shows that MLPs or even linear projections can be competitive for this problem, as long as we utilize proper data preprocessing. Concretely, we decompose the series into a **trend component** and a **seasonal component** and use individual networks to predict their future:

$$\mathbf{X}' = \mathrm{Reshape}(\mathbf{X}) \in \mathbb{R}^{N\times T} \tag{3}$$

$$\mathbf{X}_\mathrm{t} = \mathrm{AvgPool}(\mathrm{Padding}(\mathbf{X}')), \quad \mathbf{X}_\mathrm{s} = \mathbf{X}' - \mathbf{X}_\mathrm{t} \tag{4}$$

$$\hat{\mathbf{Y}}_\mathrm{t} = \mathrm{MLP}_\mathrm{t}(\mathbf{X}_\mathrm{t}) \in \mathbb{R}^{N\times S}, \quad \hat{\mathbf{Y}}_\mathrm{s} = \mathrm{MLP}_\mathrm{s}(\mathbf{X}_\mathrm{s}) \in \mathbb{R}^{N\times S} \tag{5}$$

$$\hat{\mathbf{Y}} = \mathrm{Reshape}(\hat{\mathbf{Y}}_\mathrm{t} + \hat{\mathbf{Y}}_\mathrm{s}) \in \mathbb{R}^{S\times N} \tag{6}$$

Note that here we consider different variates in the same time series as independent but use the shared network to predict them.

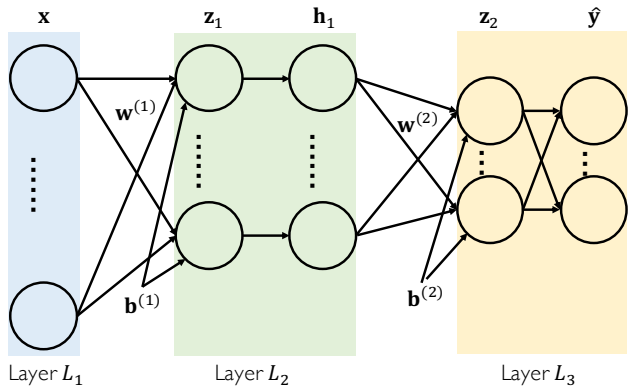

Figure 4: The network architecture of the Multilayer Perceptron (MLP).

2.2 Forward Propagation

$\mathrm{MLP_s}$ and $\mathrm{MLP_t}$ are two independent networks with the same architecture. The intermediate outputs $\mathbf{z}_1$, $\mathbf{h}_1$, $\mathbf{z}_2$, and $\widehat{\mathbf{y}}$ as the directed graph are shown below:

$$\begin{aligned}
\mathbf{z}_1 &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \\
\mathbf{h}_1 &= \mathrm{ReLU}(\mathbf{z}_1) \\
\mathbf{z}_2 &= \mathbf{W}^{(2)}\mathbf{h}_1 + \mathbf{b}^{(2)} \\
\widehat{\mathbf{y}} &= \mathbf{z}_2.
\end{aligned} \tag{7}$$

There is no activation function in the last layer since we are solving a regression problem.

2.3 Loss Function

After forward propagation, you should use the **mean squared error (MSE)** loss function:

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{mSN} \sum_{i=1}^{m} \sum_{j=1}^{S} \sum_{k=1}^{N} \left(\mathbf{y}_{j,k}^{(i)} - \hat{\mathbf{y}}_{j,k}^{(i)} \right)^2, \tag{8}$$

where m is the number of samples in each mini-batch.

2.4 Backward Propagation

Then, similar to what we have done in Part One, you can calculate the gradient of all the parameters and use (stochastic) gradient descent to update the parameters.

2.5 Task Description

We provide the starter code in `MLP` folder. To get full scores, you **only** need to complete the `model.py` marked with **TODO**. Besides, you are free to modify other parts of the code for your convenience.

To get you comfortable with gradient derivation, in this part, we will be using `numpy`. Frameworks that support auto-derivation are **not** allowed. We encourage you to adopt a **vectorized** implementation to speed up computation. The concrete tasks and their scores are as follows:

1. Implement forward propagation and backward propagation algorithms. Train the network, plot the change in loss during training, and showcase some predictions from your models. (**10 points**)
2. Train the network using proper hyper-parameters (batch size, learning rate, etc) and report the test MSE, which should not exceed 0.6. (**10 points**)

3 Part Three: Convolutional Neural Network (CNN)

In this part, you will implement a convolutional neural network [5] using PyTorch [8]. In addition to understanding the principles of CNN, we expect you to get a better grasp of PyTorch. You can learn more about the usage of PyTorch through the PyTorch Tutorial.

As we have learned in class, CNNs are made up of layers with learnable parameters, including weights and biases. Each layer takes the output of the previous layer to perform some operations and produces an output. In recent years, main-stream CNNs, such as AlexNet [6], VGG [11], GoogleNet [13], ResNet [1], DenseNet [3], EfficientNet [14] and so on, have achieved increasingly better performance on ImageNet dataset and have been leveraged to diverse applications across many fields. In this homework, a medical imaging dataset named ChestX-ray14 [16] is provided. As shown in Figure 5, ChestX-ray14 comprises 112,120 frontal-view X-ray images of 30,805 (collected from the year 1992 to 2015) unique patients with the text-mined 14 common disease labels, mined from the text radiological reports via NLP techniques. You are required to solve the problem of thorax disease image classification by modern convolutional neural networks (CNNs).

Figure 5: Example images of ChestX-ray14

3.1 Dataset

Dataset Description. The dataset contains images with 14 disease labels like "*Infiltration*", "*Effusion*", and etc. Note that these labels are not mutually exclusive, meaning that some images may have multiple labels at the same time. There is also an additional label named "*No Finding*", which means no disease is detected in this image. We will use the official split for the training and validation set.

Download Link. `https://cloud.tsinghua.edu.cn/f/be2263f9b797474b9f8c/`.

3.2 Loss Function and Evaluation Metric

The task in this dataset is multi-label binary classification. We use the **binary cross-entropy (BCE)** loss function, defined as follows:

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{mK}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \cdot \log \sigma\left(x_k^{(i)}\right) + \left(1 - y_k^{(i)}\right) \cdot \log\left(1 - \sigma\left(x_k^{(i)}\right)\right), \tag{9}$$

where $x^{(i)}$ is the output logits of neural networks and σ is the sigmoid function.

The evaluation metric is the **average AUC** for the 14 diseases.

3.3 Tutorials

- **PyTorch**
  - https://pytorch.org/tutorials/
  - https://github.com/yunjey/pytorch-tutorial

3.4 Tasks Description

Task A You are required to use the architecture of ResNet-18 to train a new model from scratch on the *training* set and then evaluate on the given *test* set.

Task B Design and implement your own convolutional neural networks (CNNs) to train a new model from scratch on the *training* set and then evaluate on the given *test* set. Your proposed model needs to be different from ResNet [1]. Some recipes can be borrowed from GoogLeNet [13], DenseNet [3], ResNeXt [17], ConvNeXt [7], MLP-Mixer [15], and etc. Good ideas and advanced structural design will be encouraged for higher scores.

To be more precise, you need to finish the following tasks:

1. Train a model **A** for Task A, evaluate its AUC on *test* set, and report training and test curves. Note that, the *start code* of this task is provided in `CNN` folder, and it is runnable; you can directly run it and report the results with "`python main.py`". (**10 points**)
2. Leverage a proper neural network visualization toolkit to visualize **conv features** or **saliency maps** of model **A**. (**10 points**)
   - **pytorch**: https://github.com/utkuozbulak/pytorch-cnn-visualizations
   - **others**: Any other toolkits if you think they are helpful...
3. Train a model **B** for Task B, evaluate its AUC on *test* set, and report training and test curves. (**10 points**).
4. Use training techniques, including but not limited to **data augmentation** and **learning rate strategies**, either sourced from existing materials or proposed by yourself, to improve the performance of your model **B** and give a detailed ablation study in your report. (**15 points**)

4 Submit Format

1. For Part One, you should submit a report (in PDF) with detailed derivation.
2. For Part Two and Three, you should submit the filled *start code* and ensure that it is runnable. Note that you also need to write a report for your experiments, which is supposed to cover the design of your model, technical details, visualization, experimental results (including training and validation curves), and the necessary references.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
[2] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. In *CVPR*, 2018.
[3] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.
[4] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
[5] D. P. Kingma and M. Welling. Auto-encoding variational bayes. In *ICLR*, 2014.
[6] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *NeurIPS*, 2012.
[7] Z. Liu, H. Mao, C. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A convnet for the 2020s. In *CVPR*, 2022.
[8] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
[9] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *IJCV*, 2015.
[10] T. Salimans and D. P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *NeurIPS*, 2016.

[11] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[12] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein. Implicit neural representations with periodic activation functions. In *NeurIPS*, 2020.

[13] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.

[14] M. Tan and Q. V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

[15] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. In *NeurIPS*, 2021.

[16] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, and R. M. Summers. Chestx-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In *CVPR*, 2017.

[17] S. Xie, R. B. Girshick, P. Dollár, Z. Tu, and K. He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017.

[18] A. Zeng, M. Chen, L. Zhang, and Q. Xu. Are transformers effective for time series forecasting? In *AAAI*, 2023.

[19] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, 2021.